



طراحی سیستم‌های دیجیتال

بهار ۱۴۰۵

استاد: دکتر اجلالی

محسن صلاح - ۴۰۳۱۰۶۲۳۸

تمرین ۵ - سوال ۲

کنترل‌کننده چراغ راهنمایی چهارراه

۱ هدف تمرین

در این سؤال، یک کنترل‌کننده چراغ راهنمایی برای چهارراهی با دو مسیر شمال-جنوب و شرق-غرب طراحی شده است. پیاده‌سازی به صورت ساختاری انجام شده؛ ماژول اصلی با یک FSM سنکرون، دو نمونه از یک ماژول داخلی چراغ را مدیریت می‌کند.

۲ توضیح معماری

طراحی از دو ماژول تشکیل شده است:

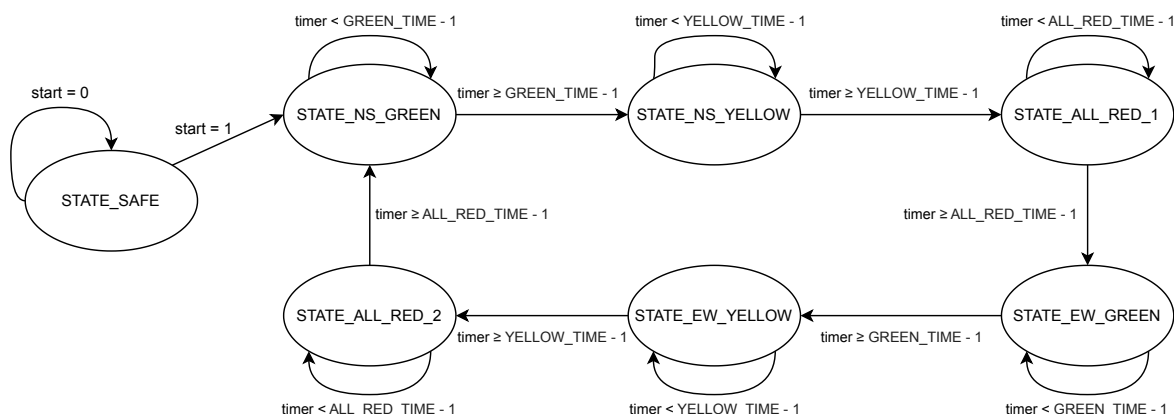
- `traffic_light`: ماژول داخلی و ترکیبی که یک دستور ۲ بیتی دریافت کرده و دقیقاً یکی از سه خروجی `red` یا `yellow`، `green` را فعال می‌کند.
- `traffic_light_controller`: ماژول سطح بالا که شامل FSM، شمارنده `timer` و دو نمونه‌ی ساختاری از `traffic_light` برای مسیرهای شمال-جنوب و شرق-غرب می‌باشد.

FSM دارای ۷ حالت است. از هر حالتی، اگر `start=0` شود، FSM در سیکل بعدی به `STATE_SAFE` باز می‌گردد. جدول انتقال حالت‌ها به این صورت می‌باشد:

حالت	کد	NS	EW	شرط انتقال	حالت بعدی
STATE_SAFE	000	قرمز	قرمز	<code>start=1</code>	NS_GREEN
STATE_NS_GREEN	001	سبز	قرمز	<code>timer ≥ GREEN_TIME-۱</code>	NS_YELLOW
STATE_NS_YELLOW	010	زرد	قرمز	<code>timer ≥ YELLOW_TIME-۱</code>	ALL_RED_1
STATE_ALL_RED_1	011	قرمز	قرمز	<code>timer ≥ ALL_RED_TIME-۱</code>	EW_GREEN
STATE_EW_GREEN	100	قرمز	سبز	<code>timer ≥ GREEN_TIME-۱</code>	EW_YELLOW
STATE_EW_YELLOW	101	قرمز	زرد	<code>timer ≥ YELLOW_TIME-۱</code>	ALL_RED_2
STATE_ALL_RED_2	110	قرمز	قرمز	<code>timer ≥ ALL_RED_TIME-۱</code>	NS_GREEN

Table ۱: چهارراه کنترل‌کننده حالت انتقال جدول

اگر `ALL_RED_TIME=0` باشد، حالت‌های `ALL_RED_1` و `ALL_RED_2` نادیده گرفته می‌شوند تا از ماندن یک سیکل اضافه در آن‌ها جلوگیری شود. دیاگرام FSM طراحی شده در شکل زیر نشان داده شده است:



شکل ۱: دیاگرام FSM کنترل‌کننده‌ی چهارراه

۳ توضیح کد

۱.۳ ماژول traffic_light

این ماژول یک decoder ترکیبی است که دستور ۲ بیتی light_state را به سه خروجی one-hot تبدیل می‌کند. حالت 2'b11 تعریف نشده؛ default آن را نیز قرمز می‌کند تا خروجی همواره معتبر باشد. کد آن به این صورت می‌باشد:

```

1 module traffic_light (
2     input wire [1:0] light_state,
3     output reg green, output reg yellow, output reg red
4 );
5     always @(*) begin
6         case (light_state)
7             2'b10: begin green=1; yellow=0; red=0; end // GREEN
8             2'b01: begin green=0; yellow=1; red=0; end // YELLOW
9             default: begin green=0; yellow=0; red=1; end // RED
10        endcase
11    end
12 endmodule

```

۲.۳ ماژول traffic_light_controller

این ماژول از دو بلوک always جداگانه ساخته شده که طبق الگوی two-process FSM کار می‌کنند. بلوک Sequential: فقط current_state و timer را مدیریت می‌کند. تشخیص تغییر حالت با مقایسه‌ی current_state و next_state انجام می‌شود؛ در صورت انتقال، timer ریست می‌شود و در غیر این صورت یک واحد افزایش می‌یابد:

```

1 always @(posedge clk) begin
2     if (rst) begin
3         current_state <= STATE_SAFE;
4         timer <= 16'd0;
5     end else begin

```

```

6         if (current_state != next_state) begin
7             current_state <= next_state;
8             timer <= 16'd0; // reset on every state transition
9         end else begin
10            timer <= timer + 1'b1;
11        end
12    end
13 end

```

بلوک Combinatorial: مقدار `next_state` و سیگنال‌های دستور چراغ را محاسبه می‌کند. مقادیر پیش فرض در ابتدا ست می‌شوند (نگه‌داشتن حالت و قرمز کردن هر دو مسیر). اگر `start=0` باشد، `next_state` مستقیماً به `STATE_SAFE` ست می‌شود؛ در غیر این صورت، `case` بر اساس حالت فعلی اجرا می‌شود:

```

1 always @(*) begin
2     next_state = current_state; // default: hold
3     ns_light_state = 2'b00; // default: RED
4     ew_light_state = 2'b00; // default: RED
5
6     if (!start) begin
7         next_state = STATE_SAFE;
8     end else begin
9         case (current_state)
10            STATE_SAFE:
11                next_state = STATE_NS_GREEN;
12
13            STATE_NS_GREEN: begin
14                ns_light_state = 2'b10; // GREEN
15                if (timer >= GREEN_TIME - 1)
16                    next_state = STATE_NS_YELLOW;
17            end
18
19            STATE_NS_YELLOW: begin
20                ns_light_state = 2'b01; // YELLOW
21                if (timer >= YELLOW_TIME - 1) begin
22                    if (ALL_RED_TIME > 0) next_state = STATE_ALL_RED_1;
23                    else next_state = STATE_EW_GREEN;
24                end
25            end
26
27            STATE_ALL_RED_1:
28                if (timer >= ALL_RED_TIME - 1) next_state = STATE_EW_GREEN;
29
30            // STATE_EW_GREEN, STATE_EW_YELLOW, STATE_ALL_RED_2
31            // follow the same symmetric pattern for the EW direction.
32
33            default: next_state = STATE_SAFE;
34        endcase
35    end
36 end

```

۳.۳ تست‌بنج

تسک **check_phase**: این تسک بر اساس فاز مورد انتظار (NS_GREEN=۱، SAFE/ALL_RED=۰)، مقادیر صحیح خروجی را از روی مشخصه (EW_YELLOW=۴، EW_GREEN=۳، NS_YELLOW=۲) محاسبه کرده و با خروجی واقعی DUT مقایسه می‌کند:

```
1 task check_phase;
2     input integer phase;
3     reg exp_ns_g, exp_ns_y, exp_ns_r;
4     reg exp_ew_g, exp_ew_y, exp_ew_r;
5     begin
6         test_num = test_num + 1;
7         // Reference model: default both directions RED
8         exp_ns_g=0; exp_ns_y=0; exp_ns_r=1;
9         exp_ew_g=0; exp_ew_y=0; exp_ew_r=1;
10        case (phase)
11            1: {exp_ns_g,exp_ns_y,exp_ns_r} = 3'b100; // NS GREEN
12            2: {exp_ns_g,exp_ns_y,exp_ns_r} = 3'b010; // NS YELLOW
13            3: {exp_ew_g,exp_ew_y,exp_ew_r} = 3'b100; // EW GREEN
14            4: {exp_ew_g,exp_ew_y,exp_ew_r} = 3'b010; // EW YELLOW
15        endcase
16        if (ns_green===exp_ns_g && ns_yellow===exp_ns_y && ns_red===exp_ns_r &&
17            ew_green===exp_ew_g && ew_yellow===exp_ew_y && ew_red===exp_ew_r)
18            // [PASS]
19        else
20            // [FAIL] with expected vs got
21        end
22    endtask
```

همچنین یک بلوک @posedge clk) always در کل شبیه‌سازی اجرا می‌شود که در هر سیکل دو شرط ایمنی را بررسی می‌کند: فعال نشدن همزمان ns_green و ew_green، و فعال بودن دقیقاً یک چراغ در هر مسیر.

تست‌کیس‌ها:

قبل از شروع تست‌های اصلی، rst اعمال شده و بررسی می‌شود که هر دو مسیر در حالت ایمن باشند:

```
1 // Test 1
2 apply_reset();
3 #2;
4 check_phase(0);
```

بلافاصله بعد از آزاد شدن rst، بررسی می‌شود که ns_red=1 و ew_red=1 باشند.

تست‌های اصلی به ترتیب زیر هستند:

```
1 // Test 2
2 repeat(5) @(posedge clk); #2;
3 check_phase(0);
```

با start=0، مدار باید در حالت ایمن بماند. پس از ۵ سیکل بدون start، هیچ انتقال حالتی نباید رخ داده باشد.

```

1 // Test 3
2 apply_reset(); start = 1'b1;
3
4 @(posedge clk); #2;
5 check_phase(1); // SAFE -> NS_GREEN
6
7 repeat(GREEN_TIME) @(posedge clk); #2;
8 check_phase(2); // NS_GREEN -> NS_YELLOW
9
10 repeat(YELLOW_TIME) @(posedge clk); #2;
11 check_phase(0); // NS_YELLOW -> ALL_RED
12
13 repeat(ALL_RED_TIME) @(posedge clk); #2;
14 check_phase(3); // ALL_RED -> EW_GREEN
15
16 repeat(GREEN_TIME) @(posedge clk); #2;
17 check_phase(4); // EW_GREEN -> EW_YELLOW
18
19 repeat(YELLOW_TIME) @(posedge clk); #2;
20 check_phase(0); // EW_YELLOW -> ALL_RED
21
22 repeat(ALL_RED_TIME) @(posedge clk); #2;
23 check_phase(1); // ALL_RED -> NS_GREEN (repeats)

```

هر انتقال حالت دقیقاً در لحظه‌ی صحیح بررسی می‌شود. مدت هر حالت مستقیماً از پارامترهای مدار خوانده می‌شود؛ مثلاً `repeat(GREEN_TIME)` سیکل برای ماندن در `NS_GREEN` و سپس `check_phase(2)` برای تأیید ورود به `NS_YELLOW` در مجموع ۷ چک صریح برای یک سیکل کامل وجود دارد.

```

1 // Test 4
2 apply_reset(); start = 1'b1;
3 @(posedge clk); // posedge 1: SAFE -> NS_GREEN
4 @(posedge clk); #1; // posedge 2: NS_GREEN; #1 steps past the edge
5 start = 1'b0; // de-assert between posedge 2 and 3 (no race condition)
6 @(posedge clk); #2;
7 check_phase(0);

```

پس از ۲ سیکل در `NS_GREEN`، با `start=0`، مدار در سیکل بعدی باید به `STATE_SAFE` برگردد. تأخیر #1 قبل از `start=0` برای جلوگیری از `race condition` با لبه‌ی ساعت است.

```

1 // Test 5
2 apply_reset(); start = 1'b1;
3 repeat(2 * 2 * (GREEN_TIME + YELLOW_TIME + ALL_RED_TIME)) @(posedge clk);

```

مدار برای ۲ سیکل کامل اجرا می‌شود و بلوک `assertion` موازی در هر `posedge` دو شرط ایمنی را چک می‌کند: فعال نشدن همزمان دو چراغ سبز، و فعال بودن دقیقاً یک چراغ در هر مسیر.

۴ اجرای کد

نتیجه‌ی اجرای کد به این صورت می‌باشد:

```
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q2
/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q2
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q2$ iverilog DSD_HW5_Q2.v DSD_HW5_Q2_tb.v
&& vvp a.out
VCD info: dumpfile DSD_HW5_Q2.vcd opened for output.

=== Traffic Light Controller Testbench ===
GREEN_TIME=3 | YELLOW_TIME=1 | ALL_RED_TIME=1

--- Test 1: Synchronous Reset & Safe State ---
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-> Expecting both RED after reset
[PASS] Check 1: NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-----

--- Test 2: Safe State Hold (start=0) ---
[Cycle 1] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 2] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 3] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 4] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 5] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-> After 5 idle cycles; still expecting both RED
[PASS] Check 2: NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-----

--- Test 3: Full Traffic Cycle (NS <-> EW rotation) ---
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 1] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
-> SAFE to NS_GREEN
[PASS] Check 3: NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
-----
[Cycle 2] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 3] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 4] NS=(G:0 Y:1 R:0) | EW=(G:0 Y:0 R:1)
-> NS_GREEN to NS_YELLOW
[PASS] Check 4: NS=(G:0 Y:1 R:0) | EW=(G:0 Y:0 R:1)
-----
[Cycle 5] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-> NS_YELLOW to ALL_RED (NS->EW transition)
[PASS] Check 5: NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-----
[Cycle 6] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
-> ALL_RED to EW_GREEN
[PASS] Check 6: NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
-----
[Cycle 7] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 8] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 9] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:1 R:0)
-> EW_GREEN to EW_YELLOW
[PASS] Check 7: NS=(G:0 Y:0 R:1) | EW=(G:0 Y:1 R:0)
-----
```

شکل ۲: خروجی تست‌بنچ در ترمینال

```

mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q2
/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q2

[Cycle 9] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:1 R:0)
-> EW_GREEN to EW_YELLOW
[PASS] Check 7: NS=(G:0 Y:0 R:1) | EW=(G:0 Y:1 R:0)
-----
[Cycle 10] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-> EW_YELLOW to ALL_RED (EW->NS transition)
[PASS] Check 8: NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-----
[Cycle 11] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
-> ALL_RED to NS_GREEN (cycle repeats)
[PASS] Check 9: NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
-----

--- Test 4: De-asserting Start Mid-Cycle ---
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 1] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 2] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 3] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-> start de-asserted mid-cycle; expecting immediate return to SAFE
[PASS] Check 10: NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
-----

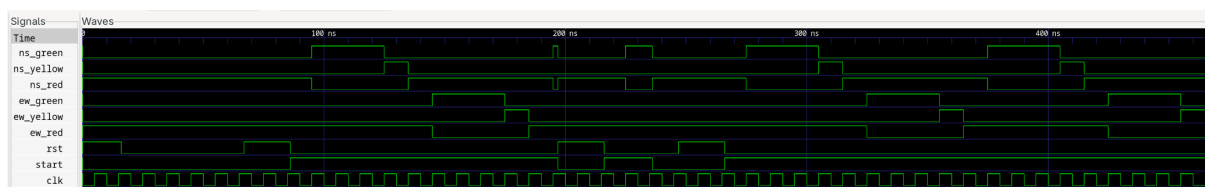
--- Test 5: Safety Assertions over 2 Full Cycles ---
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 0] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 1] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 2] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 3] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 4] NS=(G:0 Y:1 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 5] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 6] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 7] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 8] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 9] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:1 R:0)
[Cycle 10] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 11] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 12] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 13] NS=(G:1 Y:0 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 14] NS=(G:0 Y:1 R:0) | EW=(G:0 Y:0 R:1)
[Cycle 15] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:0 R:1)
[Cycle 16] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 17] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 18] NS=(G:0 Y:0 R:1) | EW=(G:1 Y:0 R:0)
[Cycle 19] NS=(G:0 Y:0 R:1) | EW=(G:0 Y:1 R:0)
-> Safety check complete (violations, if any, shown above)

=== 10 PASSED, 0 FAILED out of 10 explicit checks ===
DSD_HWS_Q2_tb.v:220: $finish called at 465000 (1ps)
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q2$

```

شکل ۳: خروجی تست‌بنچ در ترمینال (ادامه)

همان طور که مشاهده می‌شود، تمام ۱۰ تست PASS شده‌اند.
شکل موج سیگنال‌ها در GTKWave نیز به این صورت می‌باشد:



شکل ۴: شکل موج سیگنال‌ها در GTKWave

همان طور که مشاهده می‌شود، ترتیب حالت‌ها و زمان‌بندی چراغ‌ها با جدول معماری طراحی شده مطابقت دارد.